Cloud Computing

UNIT-3

Python for Amazon Web Services

- Boto is a python package that provides interface to Amazon web services(AWS).
- Set of aws services supported by boto are as follows:
- Compute Services
  1. Amazon EC2
  2. Amazon EMR
  3. AutoScalling
  4. Content Delivery
  5. Amazon CloudFront
- Database
  1. Amazon RDS
  2. Amazon DynamoDB
  3. Amazon Simple DB
  4. Amazon Elastic Catch
  5. Amazon RedShift

Python for Amazon Web Services

- Deployment and Management
  1. AWS Elastic Beanstalk
  2. AWS CloudFormation
  3. AWS Data Pipeline
- Identity and Access
  1. AWS Identity and Access Management(IAM)
- Application Services
  1. Amazon Cloud Search
  2. Amazon Simple Workflow Service(SWF)
  3. Amazon SQS
  4. Amazon SES
  5. Amazon SNS
- Monitoring
  1. Amazon CloudWatch

Python for Amazon Web Services

- Networking
  1. Amazon Route53
  2. Amazon VPC
  3. Elastic Load balancing
  4. Payments and Billing
  5. Amazon Flexible Payment Service(FPS)
- Storage
  1. Amazon Simple Storage Service(S3)
  2. Amazon Glacier
  3. Amazon Elastic Block Store(EBS)

Python for Amazon Web Services

1. Amazon EC2

- It is an ***IaaS*** service from amazon web services.
- It delivers scalable, ***pay-as- you-go*** compute capacity in the cloud.
- It provides computing capacity by launching virtual machines in Amazon ***cloud computing environment***.

Handling Amazon EC2 Instance with Python.

- Connecting to EC2 instance is first established by calling boto.ec2.connect_to_region by passing EC2 region, AWS access key and AWS secret access key.
- New instance is launched by using the conn.run_instnaces function by passing AMI-ID, instance type, EC2 key handle and security groups.
- This function returns a reservation.
- The instances associated with the reservation are obtained using reservation.instances.

Python for Amazon Web Services

- The status of an instance associated with a reservation is obtained using the instance.update function
- conn.get_all_instances function is used to get information on all running instances and it returns the reservations.
- conn.stop_instnaces function is used to stop the running instnaces.
- conn.start_instances is used to start the stop instnaces.

Amazon AutoScaling

- Amazon autoscaling allows automatically scaling of Amazon EC2 capacity up or down.
- By using autoscaling users can increase the number of EC2 instances running their applications seamlessly during spikes in the application workloads to meet the application performance requirements and scale down capacity when the workload is low to save costs.
- Connection to autoscaling service is established by calling the function boto.ec2.autoscale.connect_to_region.

Python for Amazon Web Services

- A new launch configuration is created by calling function conn.create_launch_configuration.
- Launch configuration contains instructions on how to launch new instances including the AMI-ID,instance type,security groups etc.
- After creating the instance it is associated with a new auto scaling group.
- Auto scaling group is created by using the function called conn.create_auto_scaling_group.
- The settings for Auto Scaling group such as the maximum and minimum number of instances in the group, the launch configuration, availability zones, optional load balancer to use with the group etc.
- After creating the group the policies for scaling up and scaling down are defined.
- After defining policies create a Amazon CloudWatch alarms that trigger these policies.

Python for Amazon Web Services

- To delete AutoScaling group all the instances in the group must be terminated by calling a function group.shutdown_instances.
- To delete group by calling group_to_delete.delete() function.

Amazon S3

- It is an online data storage infrastructure for storing and retrieving any amount of data.
- S3 provides highly reliable, scalable, fast, fully redundant and affordable storage infrastructure.
- To establish a connection to s3 service is by calling boto.connect_s3 function by passing aws access key and aws secret key.
- To upload the file at the specified path in the s3 bucket by calling function upload_to_s3_bucket_path.
- To upload the file to the s3 bucket root by calling the function upload_to_s3_bucket_root.

Python for Amazon Web Services

Amazon Relational Database Service (RDS)

- Amazon RDS is a web service that allows you to create instanaces of MySql , oracle, or Microsoft SQL Server in the cloud.
- With RDS, Developers can easily setup, operate and scale a RDS in the cloud.
- A connection to RDS is established by calling boto.rds.connect_to_region function.
- Launch a new database instance by calling a conn.create_dbinstance function.
- The input parameters to this function are like the instanceID, database size, instance type, database username, database password, database port, database engine, database name, security groups.
- If the status of the instance becomes available then it prints details like instnace ID, create time, instnace end point etc.

Python for Amazon Web Services

- To get all the database instances from the database by using conn.get_all_dbinstances() function.
- MySQLdb is a special package to connect from python to mysql.
- MySQLdb.connect is a function to connect to MySQL RDS instance by passing endpoint hostname, database user name, password and port number.
- conn.cursor is a function to get the cursor of database.
- cursor.execute is a function to execute the SQL commands.

Python for Amazon Web Services

Amazon DynamoDB

- It is a fully-managed, scalable, high performance No-SQL database service.
- boto.dynamodb.connect_to_region is a function to establish connection to DynamoDB by passing Region, and keys.
- After conneting to Dynamo DB a schema for the new table is created by calling conn.create_schema.
- Schema includes the hash key and range key names and types.
- Table is created by calling conn.create_table function with the table schema, read units and write units as input parameters.

Python for Amazon Web Services

Amazon DynamoDB

- conn.get_table is called to retrieve an existing table.
- New table item is created by calling table.new_item function.
- The data item is finally comitted to DynamoDB by calling item.put.
- To read data from the table table.get_item is a function.

Python for Amazon Web Services

Amazon SQS

- Amazon offers a highly scalable and reliable hosted queue for storing messages as they travel between distinct components of applications.
- boto.sqs.connect_to_region is function to establish a connection to SQS service by passing AWS region, access key and secret key.
- conn.create_queue is a function to create a new queue with queue name as input parameter.
- To know all existing queues conn.get_all_queues is a function.
- queue.write is a function to write into the queue by passing message as input parameter.
- queue.read is function to read message from the queue.

Python for Amazon Web Services

Amazon Elastic MapReduce (EMR)

- Amazon EMR is a web service that utilizes Hadoop framework running on Amazon EC2 and S3.
- Amazon EMR is suitable for massive scale data processing for applications such as data mining, data warehousing, scientific simulations etc.
- boto.emr.connect_to_region is function to connect to EMR service by passing AWS region,access key and secret key.
- After connect to EMR service a job flow step is created.
- There are two types of steps to create a job flow such as streaming and customer jar.
- To create a streaming job an object of the streaming Step class is created by specifying the job name, location of the mapper, reducer, input and output.

Python for Amazon Web Services

- The job flow is then started using the conn.run_jobflow function by passing streaming step object as a parameter.
- After completion of MapReduce job the output of the job is obtained from the output location on the s3 bucket specified while creating the streaming step.

Thank you